Cloud Workload Forecasting & Analysis

Atomity AI/ML Take-Home Assignment | Technical Report

1. Executive summary

This study analyzes a curated Microsoft Azure Public Dataset V2 trace and forecasts the active virtual-machine (VM) population and average fleet-level VM CPU utilization over a two-hour horizon. The release comprises 10,000 VM traces, each sampled 134 times at five-minute intervals (1,340,000 CPU records). After constructing timestamp-level targets, the first 110 observations were reserved for model development and the final 24 timestamps for an untouched holdout evaluation.

In three expanding-window validation folds, a 12-observation moving-average CPU baseline achieved mean absolute error (MAE) of **0.1784 CPU percentage points** and root mean squared error (RMSE) of **0.2243 percentage points**, outperforming persistence (MAE 0.2082) and a direct 24-step Ridge regression (MAE 0.3778). The frozen moving-average model generated 24 CPU predictions of **9.21637%** for timestamps 33000–39900. Final holdout CPU error was **MAE 0.1230 pp, RMSE 0.1613 pp**, or **1.327% normalized MAE** relative to actual mean holdout utilization. VM-count persistence forecast **10,000 VMs** throughout and had zero holdout error, consistent with the dataset's fixed VM population.

Operationally, fleet-average CPU alone should not drive automated capacity reductions. Segment differences and the approximately **1.272%** average training-window share of VMs with CPU at or above 80% indicate localized contention risk despite a low fleet mean. The recommendation is to maintain current VM-count capacity over the short horizon, monitor high-CPU tail metrics, and treat a validation-error-based headroom threshold as an initial alert heuristic rather than a calibrated service-level guarantee.

2. Dataset, preprocessing, and target definitions

The curated data provides CPU readings with fields `timestamp`, `vm_id`, `min_cpu`, `max_cpu`, and `avg_cpu`, supplemented by VM metadata, including VM category, virtual-core bucket, memory bucket, and reported creation/deletion timestamps. The CPU table contains **1,340,000 records**; metadata contains **10,000 VM records**. There are **134 distinct timestamps**, from 0 to 39900 inclusive, each separated by **300 seconds**. Data quality checks identified **no missing CPU or metadata fields** and **no duplicate `(vm_id, timestamp)` pairs**. Every timestamp contains readings for all **10,000 VMs**, and each VM contributes exactly 134 observations.

Two targets are defined per timestamp: (1) `active_vm_count`, the number of distinct VM IDs with a recorded CPU reading; and (2) `aggregate_cpu_utilization`, the arithmetic mean of `avg_cpu` across represented VMs, in percentage units. No reading-level missing-value imputation or gap filling was required. This CPU target is *not* core-weighted; it should not be interpreted as percent utilization of total physical CPU capacity.

The complete target index is constructed first and then split chronologically: observations 1–110 (timestamps 0–32700) are available for model selection; observations 111–134 (timestamps 33000–39900) form the 24-step, two-hour holdout. Exploratory time-series analysis, validation features, and model selection use only the first 110 timestamps. The final holdout readings are accessed separately, after forecasts have been frozen, for evaluation only.

VM lifetimes/churn: The metadata provides VM creation and deletion fields for contextual review, but the observed challenge trace has **zero changes in the represented VM population** throughout the training window. The count forecast therefore evaluates stability in the curated sample, not the ability to anticipate new VM arrivals or departures in a live cloud environment. Metadata lifetimes cannot be assumed to imply observable in-window churn.

3. Exploratory workload analysis

Within the 110-observation training window, average fleet VM CPU utilization was **9.4141%**, with a timestamp-level minimum of **9.1116%** and maximum of **9.9122%**. The highest observed training-window fleet mean occurred at timestamp **16500** (**9.9122%**); the lowest occurred at **32400** (**9.1116%**). The short trace (~9.2 hours of training) does not support robust claims about daily or weekly seasonality.

The mean share of VM readings with `avg_cpu >= 80%` was **1.2721%** (approximately 127 of 10,000 VMs at a typical timestamp). The Pearson correlation between fleet mean CPU and high-CPU VM share was approximately **0.520**: somewhat higher fleet means tended to co-occur with more individually busy VMs, but the fleet average does not fully describe tail pressure.

Utilization differed materially by metadata category. Training-window mean VM CPU was approximately **11.51% for Interactive**, **10.35% for Unknown**, and **8.26% for Delay-insensitive** VMs. Core and memory buckets also differed, although some buckets contained very few VMs; their estimates should not be treated as equally reliable. These patterns motivate segment-sensitive operational monitoring even if the aggregate forecast remains simple. The repository's training-only EDA script generates plots of fleet CPU over time, VM-level CPU distribution, high-CPU VM share, and CPU grouped by category, core bucket, and memory bucket.

4. Forecasting methodology and validation

The objective is to generate all **24 five-minute-ahead observations at once**, matching the two-hour production decision horizon. Three baseline methods were compared: (i) persistence, repeating the last value; (ii) a 12-observation moving average, repeating the mean of the previous hour; and (iii) a linear-trend extrapolation based on the latest 24 readings. A direct multi-output **Ridge regression** alternative used origin-available lag statistics and a fixed regularization configuration to predict a 24-value trajectory. At no point are actual values from forecasted future steps fed back into the model.

An **expanding-window** validation scheme used identical 24-step evaluation intervals for every model: **38 train / 24 validate; 62 train / 24 validate; 86 train / 24 validate**. All validation observations fall within the 110-timestamp public training window. The 24-point holdout remains excluded from hyperparameter choice and model selection. A limitation is the very small number of supervised direct-forecast examples in the first Ridge fold, which may contribute to its instability.

Model selection: Persistence was retained for `active_vm_count` because it exactly predicted the flat 10,000-VM training population. For fleet-average CPU, the 12-point moving average achieved the best mean validation MAE and RMSE. No final-holdout feedback was used to revise this decision.

5. Validation and holdout results

CPU model	Validation MAE (pp)	Validation RMSE (pp)
Persistence	0.2082	0.2566
Moving average, 12 observations	0.1784	0.2243
Linear trend, 24 observations	0.3656	0.4336
Direct multi-output Ridge	0.3778	0.4257

The moving average improves mean validation MAE by approximately **14.32% relative to persistence**; normalized training-window validation MAE is approximately **1.89%** using mean CPU over the first 110 timestamps as the scale denominator. Linear trend error is much larger, and Ridge fails to improve on the simple moving-average baseline in any of the three folds. The validation evidence does not support extra model complexity for this short trace.

The frozen model generates a constant **9.2163695%** fleet-average CPU forecast and **10,000** active VMs for all 24 holdout timestamps. Comparison with actual final-holdout readings gives:

Target	MAE	RMSE	Normalized MAE
VM count	0	0	0%
Fleet CPU	0.1230 pp	0.1613 pp	1.327%

The holdout's actual mean VM CPU ranged from **8.983613%** to **9.635990%**; mean forecast bias (`predicted - actual`) was **−0.052618 pp**. The final holdout happened to be easier than the average validation fold. It spans only two hours, so this result should not be extrapolated to longer horizons or other workload regimes.

6. Error analysis, peaks, and failure modes

During expanding-window validation, peak periods were defined as validation timestamps exceeding the **90th percentile of the corresponding fold's historical training CPU series**. In the first two folds, there were 10 and 6 qualifying timestamps, respectively; the third fold had none. Persistence peak MAE was **0.3904 pp** and **0.0730 pp** in folds 1 and 2, versus **0.3953 pp** and **0.1048 pp** for the moving average. Thus, while the moving average performed better overall, persistence had lower peak-period error for the limited qualifying periods. The training-derived threshold for the final holdout was **9.668362%**, above the highest actual holdout value (**9.635990%**), yielding **zero qualifying holdout peaks**. Peak holdout MAE is therefore **undefined**, not zero.

Constant 24-step predictions do not anticipate sudden jumps, changing trends, or short-lived demand spikes. The flat active-VM-count trace makes churn forecasting untestable. Core-unweighted average CPU masks variation in machine sizes and localized saturation; high-CPU share and segment metrics are necessary complements. In addition, the small history prevents confident daily-seasonality modeling or well-calibrated probabilistic intervals. Errors should be monitored in deployment, and the forecasting policy revisited after longer and more variable traces become available.

7. Operational headroom recommendation

Proposed decision rule: Keep capacity planning for the represented **10,000 active VMs** unchanged over the next two hours; use the aggregate CPU prediction for monitoring and anomaly triage rather than for automatic deprovisioning. Set an *illustrative* fleet-mean CPU review threshold at **forecast + 2 × validation RMSE**, or approximately **9.665%** (`9.21637 + 2 × 0.224295`). This is a heuristic buffer in CPU **percentage points**, **not** a calibrated prediction interval. Also track the proportion of VMs with average CPU above **80%**, and review VM-category and resource-bucket concentrations before scaling individual pools.

If monitoring repeatedly exceeds the heuristic aggregate threshold or high-CPU share increases materially, investigate saturated VM groups and provision targeted headroom. Avoid lowering reserved capacity purely because the cross-VM average appears low. The trade-off is explicit: aggressive reductions may save capacity but risk latency or local overload; conservative headroom increases cost but reduces the risk from unforeseeable spikes. Any automatic scaling trigger should be calibrated against a longer, representative trace and operational service-level objectives.

8. Reproducibility and limitations

The repository contains scripts for dataset inspection, training-window EDA, baseline and Ridge temporal validation, deterministic forecast generation, and separate holdout scoring. The forecast file contains

exactly 24 chronological rows in the required `timestamp,active_vm_count,aggregate_cpu_utilization` order. The core training/forecast pipeline should be runnable from the fixed Atomity `data-v1` release after installing declared dependencies. Automated tests check forecast shape, cadence, numerical validity, and intended recent-history use.

Key limitations are the ~9.2-hour training window, the constant represented VM population, limited number of rolling-origin folds, the unweighted CPU target, small segment buckets, a non-calibrated headroom heuristic, and a single two-hour holdout. Future work should prioritize longer traces, true VM arrivals/departures, core-weighted capacity targets, segment-level forecasts, and uncertainty calibration through additional temporal backtests.

Source: Atomity forecasting-challenge assignment and its fixed `data-v1` Azure Public Dataset V2 release. All numerical findings above are from training-only EDA/validation or the expressly designated final holdout evaluation, as labeled.